



算法与算法方法- 迭代算法



- 学习目的：
 - 通过具体示例了解算法设计的基本策略
- 学习要点：
 - 理解复杂算法背后的简单原理
- 人们利用计算机求解问题所设计的求解算法各不相同，算法设计没有固定的方法可循。但通过大量的实践，人们也总结出某些共性的规律。包括迭代法、穷举法、分治法、贪心法、回溯法等，我们这章只选择基本的、典型的算法进行讲解和学习。

问题描述： 写一个程序，能够将用户输入的三个整数中值最大的数输出。

算法设计：

数据集合中找最大元，基本思路是两两比较，像选拔赛的机制，不断留下更大的，直至没有对手。需比较次数为 $n-1$ ， n 为集合中元素数。

```
if (a<b) max=b;  
else max=a;  
if (max<c) max=c;  
else ;
```

提示： 保持if-else结构完整是逻辑严密的基础

问题描述：输入多个整数（不能有-1），以-1表示结束，要求程序求出所有输入整数（不包括-1）中的最大值，打印输出。

算法设计：

- ✓ 求最大值的算法都是两两比较，像选拔赛一样不断留下更大的，直至没有对手。

```
if (a<b) max=b; else max=a;  
if (max<c) max=c; else ;
```
- ✓ 因为整数的个数未知，需要引入循环结构进行重复的比较操作，比较的数据变量变为每次读入的新数a和当前较大值max。
- ✓ 出现的问题是第一次比较，max还没有值，怎么办？

定义: 算法是一个有限步骤的求解序列, 用来解决一类特定的问题。

例: 描述在一个有限整数序列 $\{a_n\}$ 中寻找最大值的算法。

解: 算法步骤如下:

1. 设置最大值 $\max$ 初始值, 为序列中第一个整数 a_1 .
2. ***for $i=2..n$***
 if ($\max < a_i$) then $\max = a_i$
3. ***return $\max$ { $\max = \max(a_n)$ }***

- **输入:** 算法应当有指定集的输入值.
- **输出:** 对应输入值, 算法应给出指定集的输出值, 即问题的解.
- **正确性:** 算法应针对每一组输入都能产生正确的输出.
- **有限性:** 算法应对任何输入值都能在有限步后给出结果.
- **可执行性:** 算法的每一步都能在有限时间内正确执行.
- **通用性:** 算法应可以适用既定问题的所有情况.

- 很多复杂问题，不能找到直接的求解公式；只能通过把问题归约到更小输入的问题实例来解决，或者说 $f(n)$ 的计算依赖于 $f(n-k)$ 或 $f(n/k)$ 的求解。
 - 例如： $\max(a)=a$;
 - $\max(a,b) = b > \max(a) ? b : \max(a)$;
 - $\max(a,b,c) = c > \max(a,b) ? c : \max(a,b)$;
 - ...
 - $\max(a_1, a_2, \dots, a_n) = a_n > \max(a_1, a_2, \dots, a_{n-1}) ? a_n : \max(a_1, a_2, \dots, a_{n-1})$;

? : 条件运算符

上例中运算表达式 $\text{min} = (n > m) ? m : n;$

含义是 if (n > m)

 min = m;

else

 min = n;

条件运算符?:的C语法是

(条件表达式) ? 表达式1 : 表达式2

运算规则是：当条件表达式为真，运算结果为表达式1，否则为表达式2.

- $\text{gcd}(a,b) = (a==0)?b:\text{gcd}(b\%a, a);$
- 算法 $f(n)$ 的计算公式，依赖于较小规模的问题实例 $f(n-k)$ 或 $f(n/k)$ 的求解，这种计算公式叫做递归定义。
- 递归定义在计算机中实现时，有两种方法：迭代算法，和递归函数算法。
- 迭代算法的基本思想：从 $f(n)$ 在已知一个数据或多个数据的计算开始，逐步使用递归公式求得较大或较多数据的结果，直到求得最终结果，这样的算法过程称为迭代算法。

- 使用迭代算法编写程序的基本方法：
 - 寻找问题求解的递归公式（迭代公式）；
 - 设置迭代求解的初始值；
 - 设计循环结构，实现一步步的从小到大的迭代计算。

问题：求 e^x

问题描述：用泰勒级数求 e 的 x 次幂， x 为所有实数。

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} \quad \text{for all } x$$

思路：级数的特点是多次循环，直到小于限定的误差值。所以，设计时最好用double类型，支持较小的误差范围。

e的实数幂算法设计:

Begin

定义double变量 $x, \text{sum}, \text{item};$

定义int变量 $n;$

从键盘接受输入的实数, 存入 $x;$

设置初始值: $\text{sum}=0.0, \text{item}=1.0, n=1,$

while (item 的绝对值 $>$ 规定误差)

sum 累加 $\text{item};$

滚动计算新的 $\text{item} = \text{item} * x / n;$

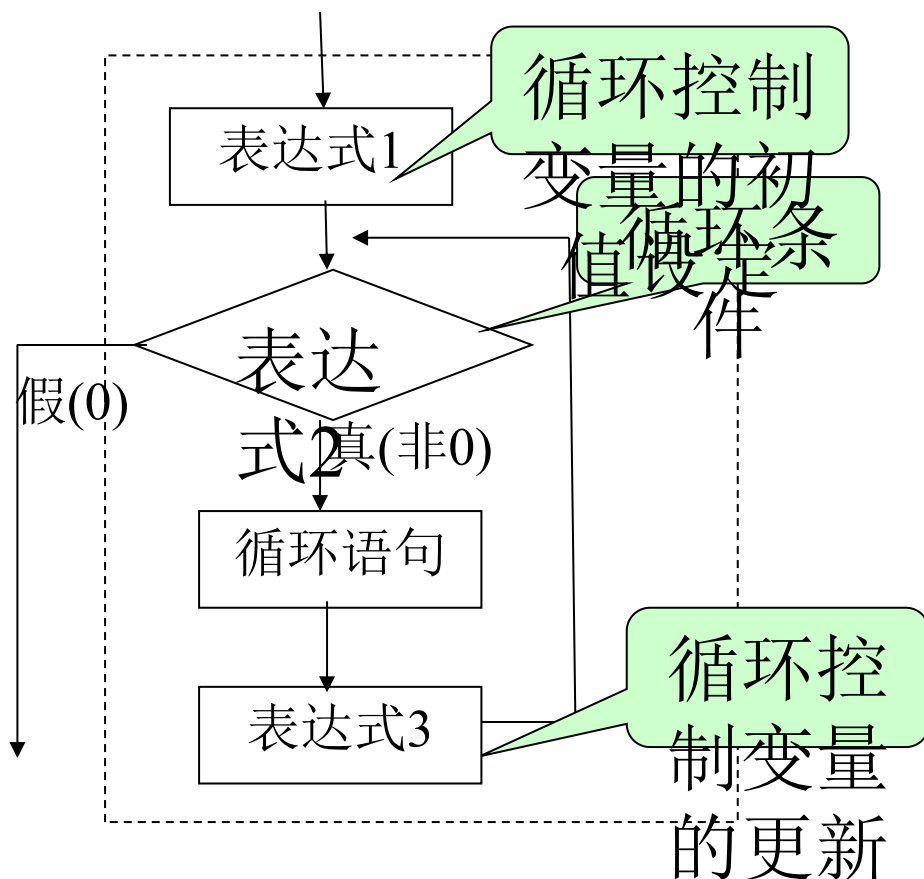
$n = n + 1;$

显示器打印" $\%.16f$ "结果值 $\text{sum};$

End

最简洁的循环结构: for循环

for(表达式1;表达式2;表达式3) 循环语句;



与仅强调循环条件的while(表达式2)循环结构相比，凸显了循环控制的全部要素：初始值、循环条件、每次循环后控制变量如何变化。

因此，for循环结构更方便程序调试和代码升级。



利用泰勒级数求 e^x

```
#include<stdio.h>
#include <math.h>
#define DIFF 1e-12
int main(void){
    double x,sum,item;
    int n;
    printf("Enter a real number for e power.\n");
    scanf("%lf",&x);
    for (sum=0.0,item=1.0,n=1; fabs(item) > DIFF; n++)
    {
        sum += item;
        item *= x/n; //为减小误差，利用前后两项滚动比
    }
    printf("exp(%f) = %.16f\n",x,sum);
    printf("math function exp = %.16f\n",exp(x));
    return 0;
}
```

表达式语句中的逗号运算符

```
for (sum=0.0,item=1.0,n=1; fabs(item) > DIFF; n++)
```

- 逗号运算符把两个表达式连接成一个表达式。
- 逗号运算符扩展了for循环的灵活性，以便在循环头中包含更多的表达式。
- 逗号运算符并不局限于在for循环中使用，但是这是它最常用的地方。
- 逗号运算符有两个性质：首先，它保证了被它分隔的表达式从左往右求值。其次，整个逗号表达式的值是最右侧项的值。

- 自加运算符++和自减运算符--是C语言特有的运算符，为了简化变量自加减1的表达式书写。
- `x++;` // 表达式`x=x+1;`的简写
- `x--;` // 表达式`x=x-1;`的简写
- 注意++和--有前缀和后缀的区别：
 - `++i; --i; /*`在使用 `i` 之前, 先使 `i` 的值加(减)
`1 */`
 - `i++; i--; /*`在使用 `i` 之后, 使 `i` 的值加(减) `1`
`*/`

✓ `double exp( double arg );`

功能: 函数返回参数 $e(2.7182818)$ 的`arg`次幂。

5. 测试和调试程序

测试各种情况，注意条件的边界一定要测，非正常情况也要测试。

测试数据1: 1;

测试数据2: 0;

测试数据3: 2.64;

测试数据4: 23.4;

测试数据5: -1;

测试数据6: -509.86.

纠正计算溢出导致的误差

Double精度内， e^x 计算过程超出极限会溢出。测试发现当 x 小于-36时近似0，可设为边界。因此，在程序中加入超出边界的处理语句。

```
#include<stdlib.h>
```

```
#define LIMIT -36
```

```
int main(void)
```

```
{    ...
```

```
    scanf("%lf",&x);
```

```
    if (x < LIMIT)
```

```
    {
```

```
        printf("exp(%f) = %.16f\n",x,0.0);
```

```
        exit(0);
```

```
    }
```

```
    for (...)
```

写出求解两个非负整数 a 和 b 的最大公约数的递归算法, 其中 $a < b$.

解: 使用递归公式 $\gcd(a,b) = \gcd(b \bmod a, a)$
和初始条件 $\gcd(0,b) = b$ 当 $b > 0$.

```
procedure  $\gcd(a,b$ : 非负整数且  $a < b$ )  
if  $a = 0$  then return  $b$   
else return  $\gcd(b \bmod a, a)$   
{输出为  $\gcd(a, b)$ }
```

```
procedure iterative gcd( $a, b$ : nonnegative integer)
if  $a=0$  then return  $b$ ;
else
     $x := a, y := b$ ;
    while  $x > 0$ 
         $r := y \% x$ ;
         $y = x$ ;
         $x := r$ ;
    return  $y$ ; {output is gcd( $a, b$ )}
```

```
#include<stdio.h>
int main(){
    int a,b,x,y,r;
    scanf("%d%d",&a,&b);
    for(x=a,y=b; x!=0; y=x,x=r){
        r=y%x;
    }
    printf("GCD(%d,%d)=%d\n",a,b,y);
    return 0;
}
```

5. 测试和调试程序

测试各种情况，注意条件的边界一定要测，非正常情况也要测试。

测试数据1: 12, 36;

测试数据2: 36, 12;

测试数据3: 22, 33;

测试数据4: 16, 20;

测试数据5: -6, -18;

测试数据6: 0, 2;

最大公约数和最小公倍数

已知两个正整数 m 和 n ，求其最大公约数和最小公倍数。

Begin

定义整数变量 a, b, x, y, r ;

从键盘读取两个正整数 a 和 b ;

$\text{for}(x=a, y=b; x \neq 0; y=x, x=r)$

$r=y\%x;$

打印 $\text{gcd}=y, \text{lcm}=a/y*b$; //先除再乘免溢出

End

练习：牛顿迭代法求方程根

- 问题描述：用牛顿迭代法求非线性方程 $f(x)=0$ 在 x_0 附近的一个实根。
- 选一个接近于 x 的真实根的近似根 x_1 (本题中取值为1)；当两次求出的根之差 $|x_{n+1}-x_n|$ 小于等于一个足够小的值(如 10^{-5})，就认为 x_{n+1} 足够接近于真实根。
- 牛顿迭代公式是： $x_{n+1}=x_n - f(x_n)/f'(x_n)$ ，其中 $f'(x_n)$ 是对 $f(x_n)$ 求一阶导数。
- 已知方程 $ax^3+bx^2+cx+d=0$ ，利用牛顿迭代法求方程根 x 在1附近的一个实根。

牛顿迭代法求方程根

```
#include <stdio.h>
#include <math.h>
#define DIFF 1e-5
int main(){
    int a,b,c,d; double x0=1, x1;
    scanf("%d%d%d%d",&a,&b,&c,&d);
    x1 = x0 - (a*x0*x0*x0 + b*x0*x0 + c*x0 + d)
/(3*a*x0*x0 + 2*b*x0 + c);
    while (fabs(x1-x0)>DIFF){
        x0 = x1;
        x1 = x0 - (a*x0*x0*x0 + b*x0*x0 + c*x0 + d)
/(3*a*x0*x0 + 2*b*x0 + c);
    }
    printf("%f\n",x1);
    return 0;
}
```

4.6 迭代算法实例

例4.5 猴子吃桃问题：有一堆桃子不知数目，猴子第一天吃掉一半，觉得不过瘾，又多吃了一只，第二天照此办理，吃掉剩下桃子的一半另加一个，天天如此，到第十天早上，猴子发现只剩一只桃子了，问这堆桃子原来有多少个？

4.6 迭代算法实例

分析：在已知 a_{10} 的情况下，可以求得 a_9 ，已知 a_9 可以求得 a_8 ……最终求得 a_1 。

$$a_9 = 2 \times (a_{10} + 1)$$

$$a_8 = 2 \times (a_9 + 1)$$

⋮

$$a_1 = 2 \times (a_2 + 1)$$

迭代公式： $a_i = 2 \times (a_{i+1} + 1)$ $i=9,8,7,6,\dots,1$

已知 a_{10} 值为1。

procedure *iterative* getPeach(*days*: nonnegative integer)

$x := 1$;

 while $days > 1$

$days := days - 1$;

$x = 2 * (x + 1)$;

 return x ; {output is peachs number}

```
#include<stdio.h>
int main()
{
    int days=10;//表示当前求解的是第几天
    int peachs=1; //表示当天的桃子数

    while (days > 1){
        days--;
        peachs = 2 * (peachs + 1);
    }
    printf("桃子数是: %d\n", peachs );
    return 0;
}
```

桃子数是: 1534

皮球从某给定高度自由落下，触地后反弹到原高度的一半，再落下，再反弹，……，如此反复。问皮球在第 n 次落地时，在空中一共经过多少距离？第 n 次反弹的高度是多少？

Begin

定义整数变量 h, n ；定义浮点变量 $dist, height$ ；

从键盘接受输入的非负整数，存入 h, n ；

$dist = (\text{double})h$; $height = h/2.0$;

for(int $i=2$; $i \leq n$; $i++$)

$dist += height * 2$;

$height /= 2.0$;

if ($n==0$) 打印 $0.0, 0.0$;

else 打印 $dist, height$;

End

任意整数的数位拆解可以用取余运算。整数 n 的数位序列 $a_k a_{k-1} \dots a_0$ 的拆解公式是： $a_i = (n / 10^i) \% 10$ 。

例：1234，67001。

1. 定义程序目标

问题：输入一个非负整数，分离出它的数位，反转后输出。提示：整数的数码位反转后得到的整数称为逆数。

样例1输入： 123456

样例1输出：

654321

样例2输入： 32000

样例2输出：

23

非负整数的逆数求解-算法1

整数逆数的算法设计:

Begn

定义一个整数变量n, reverse和weight, digit;
从键盘接受输入的一个非负整数, 存入n;

reverse=0; weight=1;

```
while(n数位分解未结束: n >= weight)
    digit = (n/weight)%10;
    reverse = reverse*10+digit;
    weight *= 10; //等同weight = weight*10;
```

显示器输出打印reverse的值;

End

/*非负整数的逆数求解-算法1*/

```
#include <stdio.h>
```

```
int main(void){
```

```
    int n=0,reverse=0,weight=1,digit=0;
```

```
    printf("Please input a nonnegative integer :\n");
```

```
    scanf("%d",&n);
```

```
    while(n >= weight){
```

```
        digit = (n/weight)%10;
```

```
        reverse = reverse*10 + digit;
```

```
        weight *= 10;
```

```
    }
```

```
    printf("reverse of %d is %d.\n",n,reverse);
```

```
    return 0;
```

```
}//算法测试： 2134567890, 出错！ Weight溢出。
```

非负整数的逆数求解-算法2

整数逆数的算法设计:

Begn

定义一个整数变量n, reverse, digit;
从键盘接受输入的一个非负整数, 存入n;

```
for (reverse=0; n!=0; n/=10 )  
    digit = n%10;  
    reverse = reverse*10+digit;
```

显示器输出打印reverse的值;

End

算法测试: 2134567890。

```
/*非负整数的逆数求解-算法2*/  
#include <stdio.h>  
int main(void){  
    int n=0,reverse=0,digit=0;  
    printf("Please input a nonnegative integer :\n");  
    scanf("%d",&n);  
    for(; n != 0; n/=10){  
        digit = n%10;  
        reverse = reverse*10 + digit;  
    }  
    printf("reverse is %d.\n",reverse);  
    return 0;  
}
```

4. 编译

目的：检查程序有效性（符合编程语言语法）。

5. 测试和调试程序

用测试数据运行程序，查看结果是否有错。

测试数据1：1234567890；

测试数据2：2134567890；

测试数据3：52000；

测试数据4：0；

测试数据5：-123；

扩展：整数的其它进制表示

```
/*整数的其他进制表示*/  
#include <stdio.h>  
int main(void){  
    int num;  
  
    scanf("%d",&num);  
    printf("八进制形式: %o\n",num);  
    printf("十六进制形式: %x\n",num);  
  
    return 0;  
}
```

思考：十进制整数 n 如何转换为2进制数？算法层面考虑.

回文是指正读和反读都一样的数或文本段。例如，12321、55555、45554、11611都是回文数。输入一个长度不超过10位的整数n，判断它是否是回文数。

Begin

```
定义一个整数变量n, reverse=0, digit;  
从键盘接受输入的一个非负整数，存入n;  
for(int tmp=n; tmp!=0; tmp/=10 )  
    digit = tmp%10;  
    reverse = reverse*10+digit;  
if (n==reverse) 打印Yes  
else 打印No
```

End

任意整数的数位拆解可以用取余运算。整数 n 的数位序列 $a_k a_{k-1} \dots a_0$ 的拆解公式是： $a_i = (n / 10^i) \% 10$ 。

例：1234，67001。

1. 定义程序目标

问题：输入一个非负整数，分离出它的数位，从高位到低位顺序输出。

样例1输入： 123456

样例1输出：

1 2 3 4 5 6

样例2输入： 32000

样例2输出：

整数分解算法1-先求最高权值

procedure *iterative* decToDigit(n : nonnegative integer)

power := 1

while $10 * power \leq n$

 power := $10 * power$; {得到10进制最高位的权值}
 } // 小心溢出

while $power \neq 0$

 print (n/power)%10);

 n := n%power;

 power := power/10;

return ; {print is $a_1 a_2 \dots a_n$ }

```
#include <stdio.h>
#define BASE 10
int main(){
    int n;
    long long power;
    scanf("%d", &n);
    for (power=1; BASE*power<=n; power*=BASE)
        ;
    for ( ; power!=0; power/=BASE){
        printf("%d ", (n/power)%BASE);
        n = n%power;
    }
    return 0;
}
```

整数分解算法2-利用计算机数据的有限性

整数分解的算法设计：

Begn

定义一个整数变量n, weight, digit;
从键盘接受输入的一个非负整数，存入n;

```
for (weight=1e9; weight!=0; weight/=10)
    if (n>=weight)
        digit = (n/weight)%10;
        打印digit;
```

End

算法测试：123456，32000。

```
/*非负整数的数位分解*/
#include <stdio.h>
#define WGT 1e9
int main(void){
    int n=0,weight=WGT,digit=0;
    printf("Please input a nonnegative integer :\n");
    scanf("%d",&n);
    for(int i=10; i>0; i--, weight/=10){
        if (n>=weight){
            digit = (n/weight)%10;
            printf("%d ",digit);
        }
    }
    return 0;
}
```

- 问题描述：已知一个正整数 n ，你要将它打印成“千分位”形式。即从个位数起，每3位之间加一个逗号。例如，将“7654321”写成“7,654,321”。
- 分析：将整数从左到右进行拆分，整数 n 的数位序列 $a_k a_{k-1} \dots a_0$ 的拆解公式是： $a_i = (n / 10^i) \% 10$ 。
- 打印每位数字时判断逗号。
 - 如果第 i 位， i 是3的倍数，应打印“, %d”
 - 需要在整数分解时添加位数的计算和判断。

```
#include <stdio.h>
#define WGT 1e9
int main(void){
    int n=0,weight=WGT,digit=0;
    scanf("%d",&n);
    for(int i=10; i>0; i--, weight/=10){
        if (n>=weight){
            digit = (n/weight)%10;
            printf("%d",digit);
            if (i%3==1 && i>1)    putchar(',');
        }
    }
    return 0;
}
```

5. 测试和调试程序

测试各种情况，注意条件的边界一定要测，非正常情况也要测试。

测试数据1: 2;

测试数据2: 123;

测试数据3: 1234;

测试数据4: 123456;

测试数据5: 123456789;

测试数据6: 1234567890;

整数 n 的数码序列和 n^2 的数码右半序列相等，称 n 为同构数。

同构数判定的算法设计：

Begin

 定义一个整数变量 n , $square$;

 从键盘接受输入的一个非负整数，存入 n ;

$square = n * n$;

 for (; ($n \neq 0$) && ($n \% 10 == square \% 10$) ;)

$n /= 10$; $square /= 10$;

 if ($n == 0$) 打印Yes;

 else 打印No;

End

练习：进制串解读

问题描述：已知一个只包含0和1的二进制数，二进制数的长度不大于10。你的任务是将其转换为十进制数并打印出来。

分析：在二进制数值系统中，最右边的数字是1位，左边的数字依次为2位、4位、8位等等。二进制数1101可解读为 $1*2^0+0*2^1+1*2^2+1*2^3$ ，即13。

复习按位解读 $a_0a_1a_2a_3$ 的逆数， $\text{reverse}(a_3)=a_3$

$\text{reverse}(a_2a_3)=a_3*10 + a_2$ $\text{reverse}(a_1a_2a_3)=\text{reverse}(a_2a_3)*10+ a_1$

二进制从高位到低位按位解读的公式

$\text{binary}(a_0a_1)=\text{binary}(a_0)*2+a_1$



```
int main(){
    int n,x;
    scanf("%d",&n);
    x=0;
    for (int w=1; n!=0; n/=10,w*=2){
        x += (n%10)*w;
    }
    printf("%d\n",x);
    return 0;
}
```

练习：二进制串解读

```
int main(){
    char binary;
    int n;
    for ( n=0; (binary=getchar())!='\n' ; ){
        if (binary>='0' && binary<='1')
            n = n*2 + (binary-'0');
    }
    printf("%d\n",n);
    return 0;
}
```

扩展：二进制小数转换

```
#include <stdio.h>
int main()
{
    char binary;
    int n, sign, p;
    double decimal=0.0;
    for ( sign=n=0, p=2; (binary=getchar())!='\n' ; ){
        if (binary=='.') sign=1;
        else if ((0==sign) && (binary>='0' && binary<='1'))
            n=n*2 + (binary-'0');
        else if (sign && (binary>='0' && binary<='1'))
        {
            decimal+=(double)(binary-'0')/p;
            p *=2;}
    }
    printf("%f\n", n+decimal);
    return 0;
}
```

练习：常用进制转换

问题描述:在内存管理中，总是会使用到二进制、八进制与十六进制数。你的任务是将十进制数 n ($0 \leq n < 256$) 转化为一个八位二进制数，一个三位八进制数与一个两位十六进制数的形式。

例如整数108分别转换为以上三种进制的数为：01101100、154、6C。 **二进制转换公式 $b_i = (n / 2^i) \% 2$**

分析：

1、十进制转二进制，除2取余，如何从高位分解？

$$(n / 2^7) \% 2 = \text{第8位}, n = n \% 2^7,$$

$$(n / 2^6) \% 2 = \text{第7位}, n = n \% 2^6$$

$$\dots (n / 2^0) \% 2 = \text{第1位}$$

迭代公式: $f(n) = \{(n/p) \% 2, f(n \% p), n \neq 0, p = 128 \rightarrow p/2\}$

```
procedure iterative decToBinary( $n$ : nonnegative integer)
power := 1
while  $2 * power \leq n$ 
    power :=  $2 * power$ ; {得到2进制最高位的权值}
while  $power \neq 0$ 
    print ( $n / power \% 2$ );
     $n := n \% power$ ;
    power :=  $power / 2$ ;
return ; {print is binary}
```

整数转二进制算法1

```
#include <stdio.h>
#define BASE 2
int main(){
    int n;
    long long power;
    scanf("%d", &n);
    for (power=1; BASE*power<=n; power*=BASE)
        ;
    for ( ; power!=0; power/=BASE){
        printf("%d ", (n/power)%BASE);
        n = n%power;
    }
    return 0;
}
```

整数转2进制算法2-利用计算机数据的有限性

十进制整数转二进制的算法设计:

Begin

定义一个整数变量n, weight, digit;
从键盘接受输入的一个非负整数, 存入n;

for (weight=64*256*256*256, i=32; i>0; i--
, weight/=2)

if (n>=weight)
 digit = (n/weight)%2;
 打印digit;

End


```
/*十进制整数转2进制算法2*/
#include <stdio.h>
#define WGT 64*256*256*256 // 2^30
#define BASE 2
int main(void){
    int n=0,weight=WGT,digit=0;
    printf("Please input a nonnegative integer :\n");
    scanf("%d",&n);
    for(int i=31; i>0; i--, weight/=BASE){
        if (n>=weight){
            digit = (n/weight)%BASE;
            printf("%d ",digit);
        }
    }
    return 0;
}
```

